

Cadenas

Programación I

UNRN

Universidad Nacional
de Río Negro

01

Setiembr

e

Programación I

Clase 07: Punteros, Arreglos y parámetros.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- **Declaración de vectores.**
- **Recorridos.**
- **Pasar vectores como parámetro.**
- **Prototipos de función.**
- **Archivos con headers.**

Punteros

- *Un puntero es una variable que contiene una dirección de memoria.*
 - Una forma de pensar la memoria es como un continuo de celdas numeradas (o direccionadas) que contienen bytes

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	206	45	4	36	247	77	138	161	228	93

Variables en memoria

- *Un puntero es una variable que contiene una dirección de memoria.*
 - Definimos 2 variables:

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	206	45	4	36	247	77	138	161	228	93
					a			b			

```
char a = 36; //a está inicializada
char b;      //b no está inicializada pero
              //hay algo (desconocido) en la memoria
```

Punteros a variables en memoria

- *Un puntero es una variable que contiene una dirección de memoria.*
 - Definimos un nuevo tipo de variable: un puntero '*'.

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	206	45	4	36	247	77	138	161	228	93
		pa			a			b		pb	

```
char a = 36;    //a está inicializada
char b;        //b no está inicializada pero
               //hay algo (desconocido) en la memoria
char *pa;      //un puntero de nombre 'pa'
char *pb;      //un puntero de nombre 'pb'
```

Punteros a variables en memoria

- *Un puntero es una variable que contiene una dirección de memoria.*
 - Definimos un nuevo tipo de variable: un puntero '*'.

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	206	45	4	36	247	77	138	161	228	93
		pa			a			b		pb	

pa y pb también
están en la
memoria.

```
char a = 36;    //a está inicializada
char b;         //b no está inicializada pero
                //hay algo (desconocido) en la memoria
char *pa;       //un puntero de nombre 'pa'
char *pb;       //un puntero de nombre 'pb'
```

Operador &

- *Un puntero es una variable que contiene una dirección de memoria.*
 - Asignamos valor a los punteros usando el operador '&' (*address of*)

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	206	45	4	36	247	77	138	161	228	93
		pa			a			b		pb	

```
char a = 36;  
char b;
```

```
char *pa;  
char *pb;
```

```
pa = &a;  
pb = &b;
```

Punteros a variables en memoria

- *¿Qué contiene cada variable?:*

```
char a = 36;  
char b;
```

```
char *pa;  
char *pb;
```

```
pa = &a;  
pb = &b;
```

```
printf("a: %d\n", a);           //valor de a  
printf("pa: %p\n", pa);        //valor del puntero pa  
printf("&a: %p\n", &a);         //valor de la dirección de memoria de a  
printf("&pa: %p\n", &pa);       //valor de la dirección de pa  
printf("*pa: %d\n", *pa);      //el contenido del puntero pa
```

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	4	36	247	77	138	161	7	93
		pa			a			b		pb	

Punteros a variables en memoria

- *¿Qué contiene cada variable?:*

```
char a = 36;  
char b;
```

```
char *pa;  
char *pb;
```

```
pa = &a;  
pb = &b;
```

```
printf("a: %d\n", a);      -> ...? 36  
printf("pa: %p\n", pa);   -> ...? 4  
printf("&a: %p\n", &a);    -> ...? 4  
printf("&pa: %p\n", &pa);  -> ...? 1  
printf("*pa: %d\n", *pa); -> ...? 36
```

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	4	36	247	77	138	161	7	93
		pa			a			b		pb	

Punteros a variables en memoria

- ¿Qué contiene cada variable?:

```
char a = 36;  
char b;
```

```
char *pa;  
char *pb;
```

```
pa = &a; //direccion de memoria de a  
pb = &b;
```

```
printf("a: %d\n", a);      -> 36  
printf("pa: %p\n", pa);   -> 4  
printf("&a: %p\n", &a);    -> 4  
printf("&pa: %p\n", &pa);  -> 1  
printf("*pa: %d\n", *pa); -> 36
```

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	4	36	247	77	138	161	7	93
		pa			a			b		pb	

Y lo mismo para b y pb. Pero como no fue inicializada no sabemos lo que va a dar. Veamos con el compilador.

Aritmética sencilla de punteros

```
char a = 36;  
char b;
```

```
char *pa;  
char *pb;
```

```
pa = &a;  
pb = &b;
```

```
//Que imprime?
```

```
printf("pa: %p\n", pa+2);
```

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	4	36	247	77	138	161	7	93
		pa			a			b		pb	

Si $pa=4 \rightarrow$

$$pa+2 = 4+2 = 6$$

y

$$*(pa+2) = \mathbf{77}$$

Pero cuidado con los
paréntesis $*pa+2$ se
resuelve como

$$= (*pa) + 2$$

$$= 36 + 2$$

$$= \mathbf{38}$$

Punteros a variables en memoria

- *¿Qué contiene cada variable?:*

```
char a = 36;  
char b;
```

```
char *pa;  
char *pb;
```

```
pa = &a;  
pb = &b;
```

```
printf("a: %d\n", a);      -> 36  
printf("pa: %p\n", pa);   -> 4  
printf("&a: %p\n", &a);    -> 4  
printf("&pa: %p\n", &pa);  -> 1  
printf("*pa: %d\n", *pa); -> 36
```

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	4	36	247	77	138	161	7	93
		pa			a			b		pb	

Usé %d para ver el valor numérico. ¿Qué pasa si uso %c?.

Fuente

```
#include <stdio.h>

int main()
{
    char a = 125;
    char b;

    char *pa;
    char *pb;

    pa = &a;
    pb = &b;

    printf("a: %d\n",a);
    printf("pa: %p\n",pa);
    printf("&a: %p\n",&a);
    printf("&pa: %p\n",&pa);
    printf("*pa: %d\n",*pa);

    printf("b: %d\n",b);
    printf("pb: %p\n",pb);
    printf("&b: %p\n",&b);
    printf("&pb: %p\n",&pb);
    printf("*pb: %d\n",*pb);

    return 0;
}
```

integer

```
#include <stdio.h>

int main()
{
    char a = 125;
    char b;

    char *pa;
    char *pb;

    pa = &a;
    pb = &b;

    printf("a: %c\n",a);
    printf("pa: %p\n",pa);
    printf("&a: %p\n",&a);
    printf("&pa: %p\n",&pa);
    printf("*pa: %c\n",*pa);

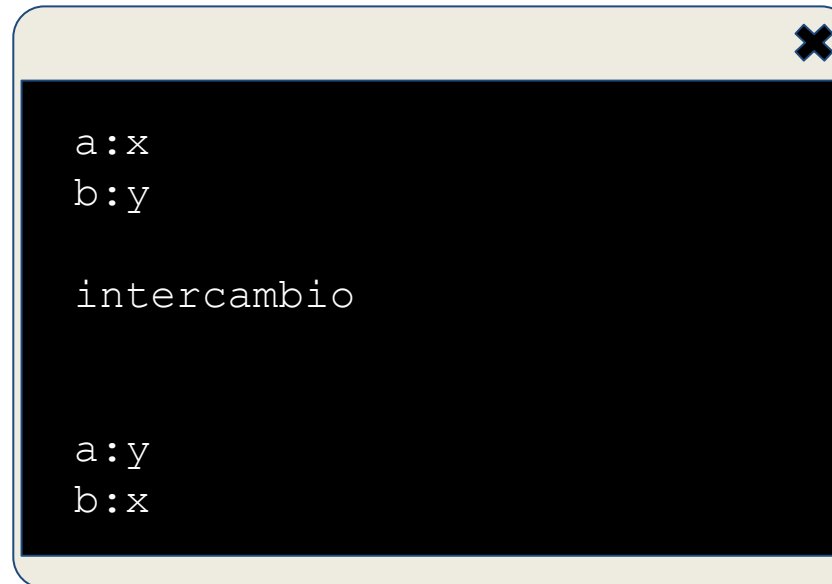
    printf("b: %c\n",b);
    printf("pb: %p\n",pb);
    printf("&b: %p\n",&b);
    printf("&pb: %p\n",&pb);
    printf("*pb: %c\n",*pb);

    return 0;
}
```

char

Punteros como argumentos: La función *swap* (*intercambiar*)

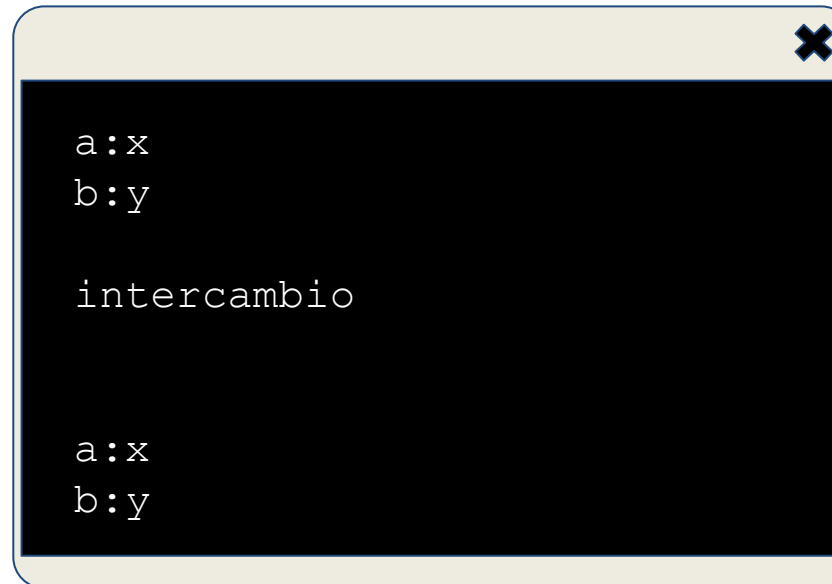
```
char a = 'x';  
char b = 'y';  
  
printf("a: %c\n",a);  
printf("b: %c\n",b);  
  
//intercambio:  
printf("intercambio:\n",a);  
...?...  
  
printf("a: %c\n",a);  
printf("b: %c\n",b);
```



```
a:x  
b:y  
  
intercambio  
  
a:y  
b:x
```

Punteros como argumentos: La función *swap* (*intercambiar*)

```
char a = 'x';  
char b = 'y';  
char temp;  
  
printf("a: %c\n",a);  
printf("b: %c\n",b);  
  
//intercambio:  
printf("intercambio:\n",a);  
temp=a;  
a=b;  
b=temp;  
  
printf("a: %c\n",a);  
printf("b: %c\n",b);
```

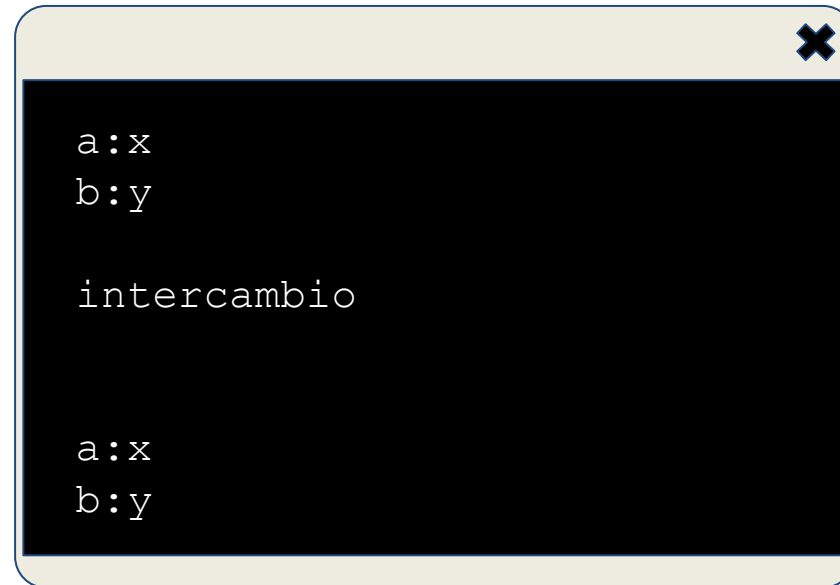


A terminal window with a black background and white text. It shows the output of the C program. The first two lines are 'a:x' and 'b:y'. Then, there is a blank line followed by the word 'intercambio'. After another blank line, the output shows 'a:x' and 'b:y' again, indicating that the values of a and b have been swapped.

```
a:x  
b:y  
  
intercambio  
  
a:x  
b:y
```

Punteros como argumentos: La función *swap* (*intercambiar*)

```
char a = 'x';  
char b = 'y';  
  
printf("a: %c\n",a);  
printf("b: %c\n",b);  
  
//intercambio:  
printf("intercambio:\n",a);  
swap(x,y)  
  
printf("a: %c\n",a);  
printf("b: %c\n",b);
```



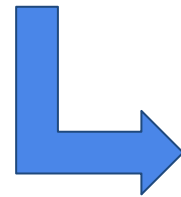
```
a:x  
b:y  
  
intercambio  
  
a:x  
b:y
```



Punteros como argumentos: La función *swap* (intercambiar)

```
void swap(char a, char b)
{
    char temp;
    temp=a;
    a=b;
    b=temp;
}
```

La función recibe parámetros por copia. Los valores de 'a' y 'b' se copian mediante la pila (stack) al volver a main, 'a' y 'b' no cambiaron su valor.



¡Punteros!
(o el pasaje por referencia en C).

Punteros como argumentos: La función *swap* (intercambiar)

```
void swap(char a, char b)
{
    char temp;
    temp=a;
    a=b;
    b=temp;
}

int main()
{
    char a = 'x';
    char b = 'y';

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    //intercambio:
    printf("intercambio:\n",a);
    swap(a,b);

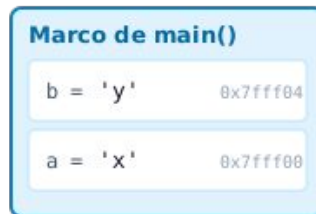
    printf("a: %c\n",a);
    printf("b: %c\n",b);

    return 0;
}
```

Direcciones ALTAS (▲)

1. Estado en main()

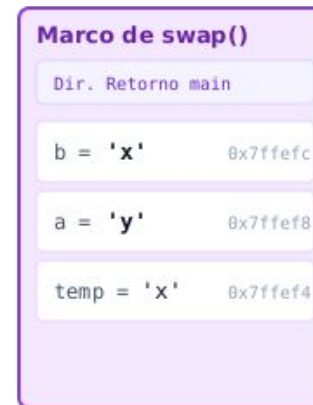
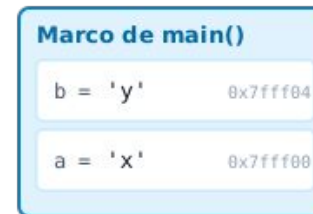
Antes de llamar a swap()



Valores originales intactos

2. Ejecutando swap()

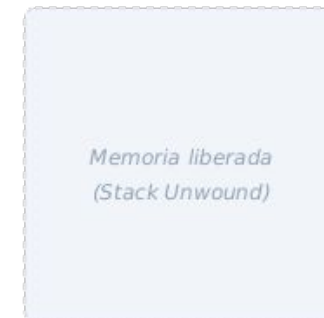
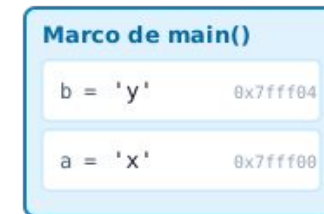
Paso por Valor (Copia en Stack)



¡Solo se cambiaron las COPIAS!

3. Retorno a main()

Frame de swap() destruido



Resultado final: SIN CAMBIOS

Direcciones BAJAS (▼)

Punteros como argumentos: La función *swap* (intercambiar)

Ahora no recibe copias sino punteros,

```
void swap(char* a, char* b)
{
    char temp;
    temp=*a;
    *a=*b;
    *b=temp;
}
```



Punteros como argumentos: La función *swap* (intercambiar)

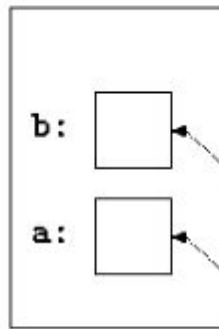
```
char a = 'x';  
char b = 'y';
```

```
printf("a: %c\n",a);  
printf("b: %c\n",b);
```

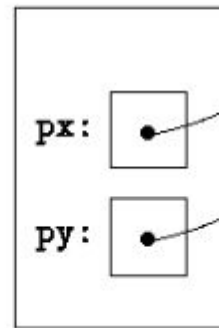
```
//intercambio:  
printf("intercambio:\n",a);  
swap(&x,&y)
```

```
printf("a: %c\n",a);  
printf("b: %c\n",b);
```

in caller:



in swap:



```
void swap(char* px, char*  
py)  
{  
    char temp;  
    temp=*px;  
    *px=*py;  
    *py=temp;  
}
```

La función *swap*: fuentes

```
#include <stdio.h>

int main()
{
    char a = 'x';
    char b = 'y';
    char temp;

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    //intercambio:
    printf("intercambio:\n",a);
    temp=a;
    a=b;
    b=temp;

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    return 0;
}
```

```
#include <stdio.h>

void swap(char a, char b)
{
    char temp;
    temp=a;
    a=b;
    b=temp;
}

int main()
{
    char a = 'x';
    char b = 'y';

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    //intercambio:
    printf("intercambio:\n",a);
    swap(a,b);

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    return 0;
}
```

```
#include <stdio.h>

void swap(char *a, char *b)
{
    char temp;
    temp=*a;
    *a=*b;
    *b=temp;
}

int main()
{
    char a = 'x';
    char b = 'y';

    printf("a: %c\n",a);
    printf("b: %c\n",b);

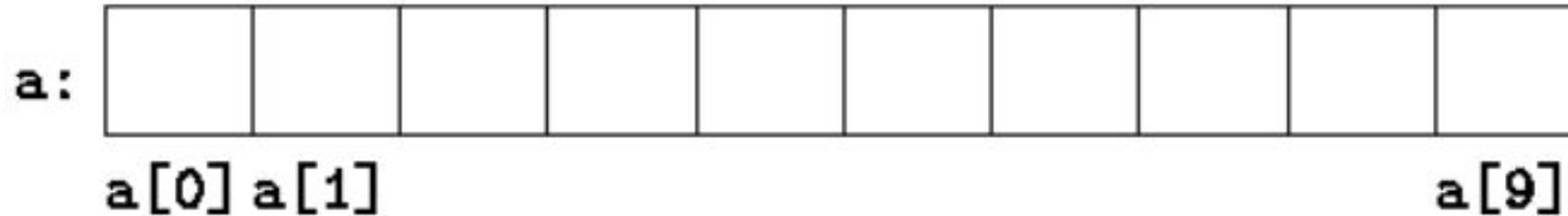
    //intercambio:
    printf("intercambio:\n",a);
    swap(&a,&b);

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    return 0;
}
```

Arreglos y punteros

- Repasemos los arreglos vistos la clase pasada
 - Declaración de arreglos:
 - `int a[10];` //arreglo de 10 enteros



Arreglos y punteros

- Repasemos los arreglos vistos la clase pasada
 - Declaración de arreglos:
 - `int a[10];` //arreglo de 10 enteros
 - `float b[10];` //arreglo de 10 flotantes
 - `char c[10];` //arreglo de 10 caracteres
 - Inicialización en la declaración.
 - `int a[5] = {10, 20, 30, 40, 50};`
 - `int a[] = {1, 2, 3, 4};`
 - `int a[100] = {0};`
 - `int a[5] = {10, 20};` // Resulta en: {10, 20, 0, 0, 0}
 - `int [6] = {[1] = 5, [4] = 20};` // Resulta en: {0, 5, 0, 0, 20, 0}

Arreglos y punteros

- Repasemos los arreglos vistos la clase pasada
 - Acceso a los elementos
 - `int a[5] = {10, 20, 30, 40, 50};`
 - `a[0]=10 a[2]=30` //con literales
 - `size_t i=3; a[i]=40` //con variables de tipo `size_t` (se puede usar `int`)
 - `for(int i=0;i<5;i++)`
`printf("a[%d]:%d\n",i,a[i]);` // imprimir el vector



```
a[0]:10
a[1]:20
a[2]:30
a[3]:40
a[4]:50
```

Arreglos y punteros

- Repasemos los arreglos vistos la clase pasada
 - Acceso a los elementos: `size_t`

```
#include <stdio.h>
#include <stddef.h>
int arr[] = {10, 20, 30, 40};
size_t n = sizeof(arr) / sizeof(arr[0]);
for (size_t i = 0; i < n; i++)
{
    printf("Posicion %zu: %d\n", i, arr[i]);
}
```

- ❖ Portabilidad y arquitectura: Garantiza el tamaño suficiente para direccionar toda la memoria disponible del sistema. Ocupa 32 bits (4 bytes) en arquitecturas de 32 bits y 64 bits (8 bytes) en sistemas de 64 bits, a diferencia de `int` que suele quedarse en 32 bits.
- ❖ Retorno de `sizeof`: El operador `sizeof` devuelve siempre un resultado de tipo `size_t`, por lo que es la opción correcta para almacenar la longitud de un arreglo.
- ❖ Seguridad semántica: Al no permitir números negativos, evita errores de frontera al utilizarlo como índice en bucles de iteración.
- ❖ Formato de impresión: En `printf`, el especificador dedicado para `size_t` es `%zu`.

Arreglos y punteros

- Repasemos los arreglos vistos la clase pasada
 - Podemos acceder con otros recorridos:
 - Pares

```
int arr[] = {10, 20, 30, 40};
size_t n = sizeof(arr) / sizeof(arr[0]);
for (size_t i = 0; i < n; i+=2)
{
    printf("Posicion %zu: %d\n", i, arr[i]);
}
```

- Impares

```
int arr[] = {10, 20, 30, 40};
size_t n = sizeof(arr) / sizeof(arr[0]);
for (size_t i = 1; i < n; i+=2)
{
    printf("Posicion %zu: %d\n", i, arr[i]);
}
```

Arreglos como parámetros

- Una función puede recibir vectores como argumentos. No se copia todo el vector como cuando se pasa un entero, ***se pasa un puntero al primer elemento***.

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	10	20	30	40	50	161	7	93

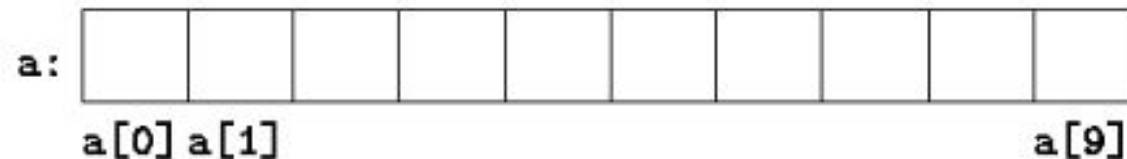
```
int a[5] = {10, 20, 30, 40, 50};  
int *pa = &a[0];
```

¿Cuanto vale `pa`?

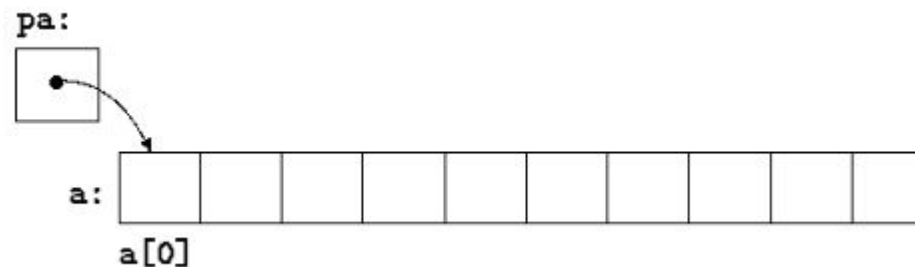
Arreglos como parámetros

- Una función puede recibir vectores como argumentos. No se copia todo el vector como cuando se pasa un entero, ***se pasa un puntero al primer elemento***.

```
int a[5] = {10, 20, 30, 40, 50};  
int *pa = &a[0];
```



¿Cuanto vale pa?



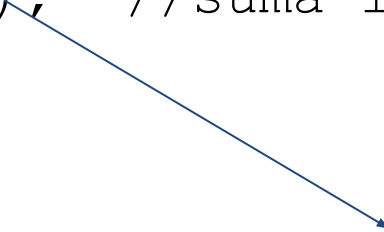
Arreglos como parámetros

- Una función puede recibir vectores como argumentos. No se copia todo el vector como cuando se pasa un entero, ***se pasa un puntero al primer elemento***.

```
int a[5] = {10, 20, 30, 40, 50};  
int suma = sumar_elementos(a); //suma los elementos del vector  
printf("%d\n", suma);
```

- ¿Qué valor imprime?

Ojo que falta completar



Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos`?

```
int sumar_elementos(int a[])  
{  
    //código de la función  
}
```

Los [] indican que es un vector y no un puntero a un entero.

```
int sumar_elementos2(int *pa)  
{  
    //código de la función  
}
```

Estas funciones son equivalentes pero la opción `a[]` es más clara para leer

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos`?

```
int sumar_elementos(int a[])  
{  
    //código de la función  
}
```



Pensemos el código
de la función en el
pizarrón

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos`?

```
int sumar_elementos(int a[], size_t nelem)
{
    //código de la función
    int suma=0;
    for(size_t i=0;i< nelem;i++)
        suma+=a[i];
    return suma;
}
```

Arreglos como parámetros

- ¿Qué pasa si modifico accidentalmente el contenido del arreglo?

```
int sumar_elementos(int a[], size_t nelem)
{
    //código de la función
    int suma=0;
    for(size_t i=0;i< nelem;i++)
    {
        suma+=a[i];
        a[i]+=suma;
    }

    return suma;
}
```

Hay formas de hacer que el compilador lo detecte antes de ejecutar: *const*



Arreglos como parámetros

- ¿Qué pasa si modifico accidentalmente el contenido del arreglo?

```
int sumar_elementos(const int a[], size_t nelem)
{
    //código de la función
    int suma=0;
    for(size_t i=0;i< nelem;i++)
    {
        a[i]+=suma; //Ahora da error de compilación!!
    }

    return suma;
}
```

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos` usando punteros?

```
//necesitando saber la cantidad de elementos.  
int sumar_elementos(int * pa, size_t nelem)  
{  
    //código de la función  
    int suma=0;  
    for(size_t i=0;i< nelem;i++)  
        suma+=* (pa+i) ;  
  
    return suma;  
}
```

Ver [filmina](#) de aritmética de punteros.

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos` usando punteros?

```
//necesitando saber la cantidad de elementos.  
int sumar_elementos(const int * pa, size_t nelem)  
{  
    //código de la función  
    int suma=0;  
    for(size_t i=0;i< nelem;i++)  
        suma+=* (pa+i);  
  
    return suma;  
}
```

Acá también vale el const.

Resumen

- Punteros
- Definición y acceso a los valores
- Algo de operaciones (aritmética).
- Parámetros por referencia.
- Arreglos
- Recorridos en arreglos con índices.
- Recorridos en arreglos con punteros.
- Suma de elementos.
- Argumentos *const*

Qué sigue

- Archivos con headers.
- Prototipos de función.